```kotlin
fun main() {  new *
    for (i in 1 ≤ .. ≤ 10) {
        println("Hello, welcome to the tutorial")
    }
    TODO( reason: "Write some cool code so that they are impressed")
}
```

Programming Tutorial - 27

9. Tutorial - Java API & Principles

Kuzey Aydınhan | 23. January 2026

Contents

1. Overview

2. Java API

3. OOP Principles

Information

- Homework 5 deadline: 29.01.

Important

Exam inspection (Einsicht) information will be published on ILIAS

Information

Exam Inspeciton

- Bring student **and** ID card
- It's not allowed to take photos or make copies
- If you have any questions or problems you can always discuss

Last Week

Generics

- Use generics by parameterizing the class
- This way can declare classes of specific types
- Reduces code duplicates

```java
class Box<T> {
    private List<T> items;

    public T getItem() {..}

    public void storeItem(T item) {..}
}

```

```java
Box<Pencil> pencilBox = new Box<Pencil>();

pencilBox.storeItem(pencil1);

```

Last Week

Bounded Generics

- Can set boundaries to type parameters
- `<T extends ...>`

```java
class NumberBox<T extends Integer> {
    private int sum;
    private List<T> items;

    public T get(int index) {..}

    public void addItem(T item) {
        items.add(item);
        sum += item;
    }
}

NumberBox<Integer> intBox = NumberBox<Integer>();

// Error: double doesnt extend integer
NumberBox<Double> doubleBox = NumberBox<Double>();

```

Last Week

Recursion

- Method that calls itself
- Has a **base** and **recursive case**

$$a_n := \begin{cases} 5, & n = 0 \\ a_{n-1} + 10, & n \neq 0 \end{cases}$$

$$(a_0 = 5, d = 10)$$

```java
int arithmeticSeq(int n) {
    // base case
    if (n == 0 ){
        return 5;
    }

    // recursive case
    return arithmeticSeq(n-1) + 10;
}

```

Motivation

- Java API
- OOP Principles

What is Java API

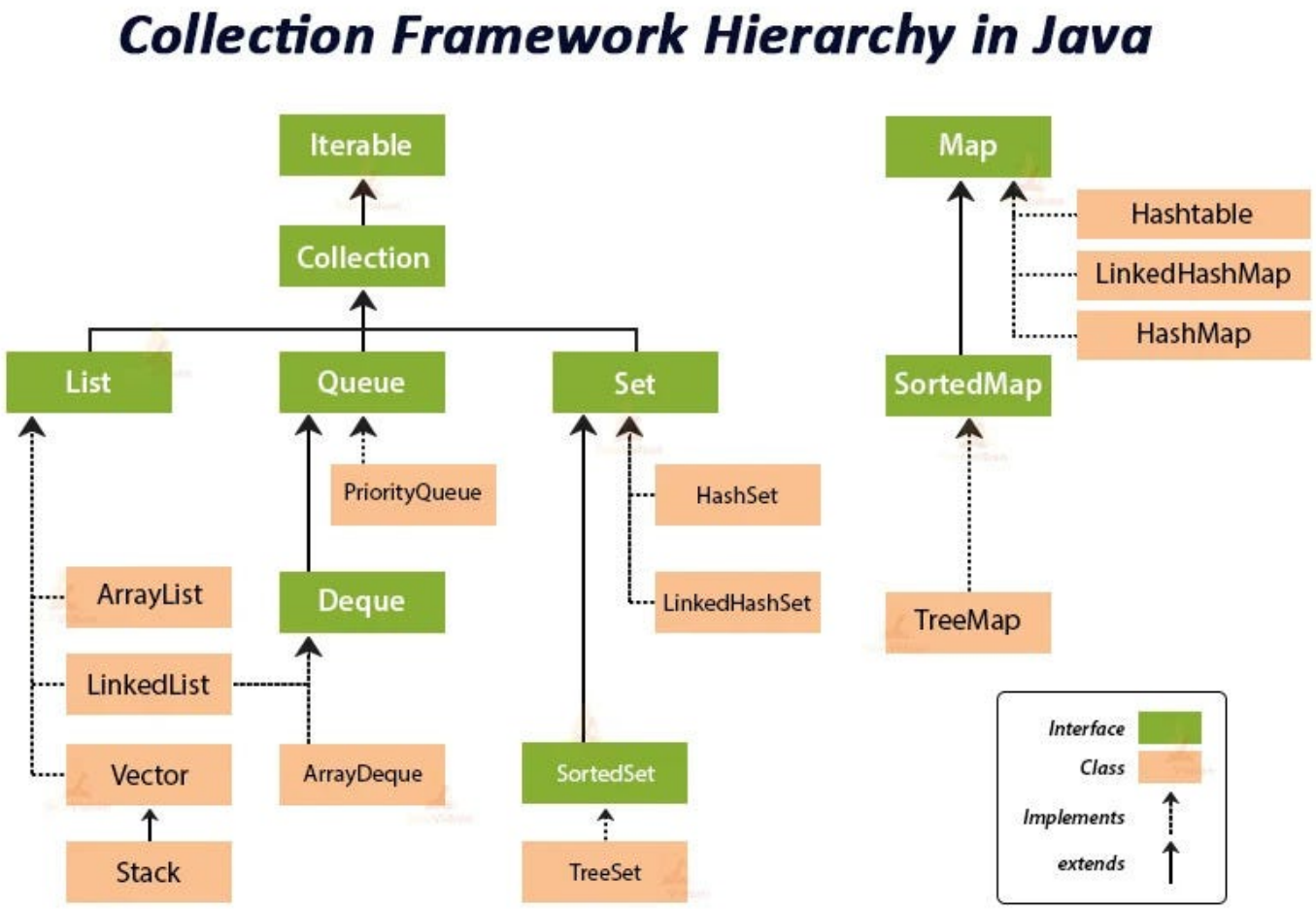
- Collection of the mostly used functionalities
- Speeds up the development process

Java API Packages

- `java.lang` (`Object, String, Enum, Iterable, Math..`)
- `java.util` (`Collections`, data structures..)
- `java.io` (`Input/Output, Streams, File Write/Read..`)

- `java.net`
- `java.security`
- `java.swing, awt..` (GUI)

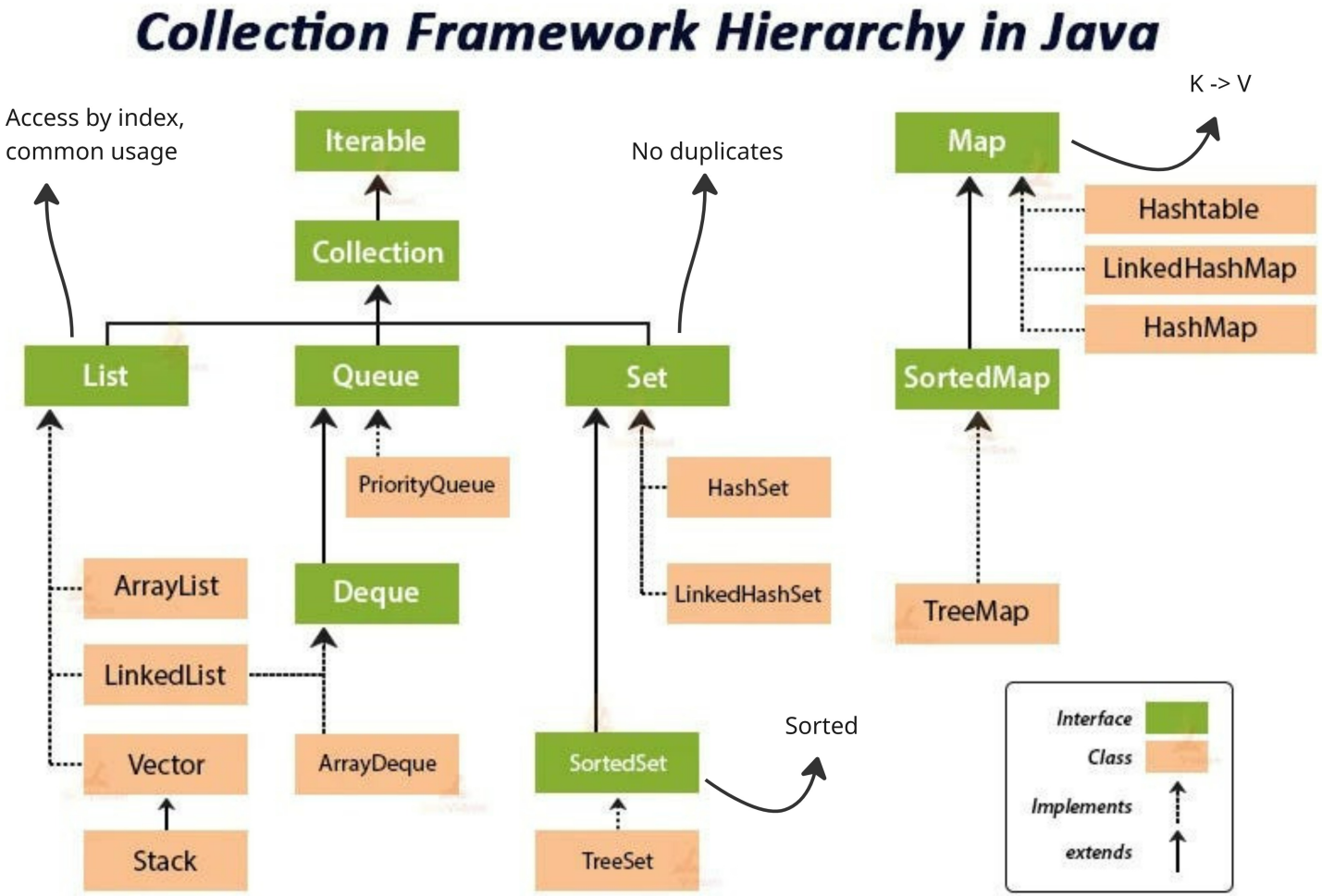
Important Ones



Collection<E>

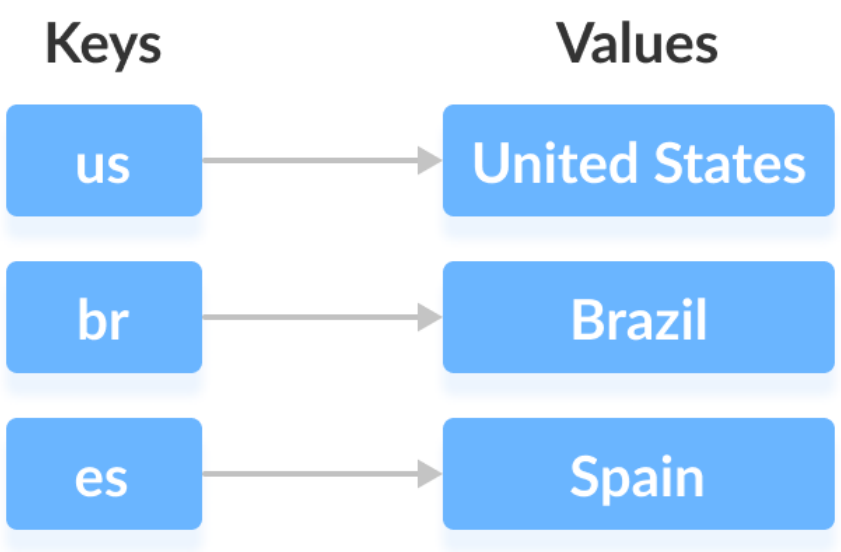
- For collection of data
- No specification
  - Sorted/not sorted?
  - Can there be duplicates?
- More specific classes: `List, Set, Queue..`

Overview


Collection Framework Hierarchy in Java
Access by index, common usage
Iterable
Collection
No duplicates
Map
K -> V
Hashtable
LinkedHashMap
HashMap
List
Queue
Set
SortedMap
PriorityQueue
HashSet
ArrayList
Deque
LinkedHashSet
TreeMap
LinkedList
Vector
ArrayDeque
SortedSet
Sorted
Stack
TreeSet
Interface
Class
Implements
extends

Map<K, V>

- Models a math function $K \rightarrow V$
- Map doesn't have duplicate keys (K)
- Each key maps to single value (V)



Map

Usage

- `put(K key, V value)`: Adds key-value pair
  - If key exists then sets the key to value (overwrites)
- `V get(K key)`: Returns the value to key
- `remove(K key)`: Removes key-value pair

```java
Map<String, Person> database = new HashMap<>();

database.put("Tony", person1);
database.put("Verstappen", person2);
database.put("Koziolek", professor);

Person result = database.get("Tony");  // -> person1

```

Map vs LinkedMap

HashMap

- Can’t control order

```
database.put("Tony", person1);
database.put("Verstappen", person2);
database.put("Koziolek", professor);

[
    "Verstappen" -> person2,
    "Tony" -> person1,
    "Koziolek" -> professor
]

```

LinkedHashMap

- Conserves insertion order

```
database.put("Tony", person1);
database.put("Verstappen", person2);
database.put("Koziolek", professor);

[
    "Tony" -> person1,
    "Verstappen" -> person2,
    "Koziolek" -> professor
]

```

SortedMap

- Sorts the items by keys
- Can customize the sort algorithm

- For example alphabetically sorted:

```
database.put("c", person3);
database.put("a", person1);
database.put("b", person2);

[
    "a" -> person1,
    "b" -> person2,
    "c" -> person3
]

```

Optional<T>

- Instead of returning `null` can use `Optional`
- Adds `null` safety
- `Optional.empty()` returns empty optional
  → similar to `null`
- `Optional.of()` wraps the item to optional
- Instead of `== null` can do `optional.isPresent`
- Use `get()` to unwrap the item

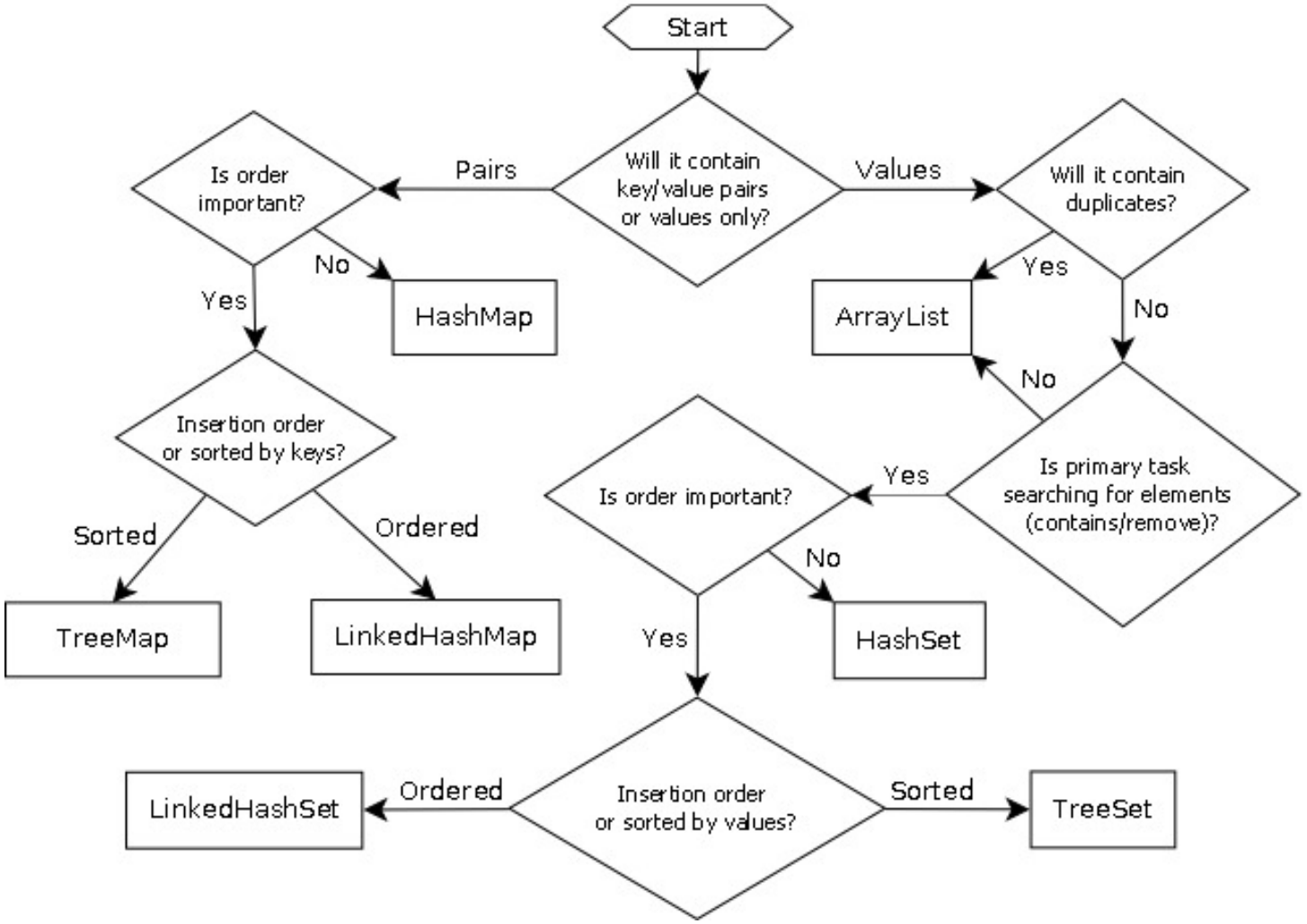
```java
Optional<Item> getItem() {
    if (itemExists()) {
        return Optional.of(item);
    }
    return Optional.empty();
}

Optional<Item> foundItemOpt = getItem();

if (foundItemOpt.isPresent) {
    Item foundItem = foundItemOpt.get();
    foundItem.use()
}

```

Java Map/Collection Cheat Sheet



Exercise 1

- Implement `Module.enrollStudent(student)` with duplicate prevention.
- Implement `StudentDatabase.getStudent(id)` returning `Optional`.
- Implement `StudentDatabase.addStudent(student)` preventing duplicates by ID.

```java
class StudentDatabase {
    private Map<String, Student> students;

    public Optional<Student> getStudent(String id) {
        // TODO: Implement me
    }

    public boolean addStudent(Student student) {
        // TODO: Implement me
    }
}

```

```java
class Module {
    private final String id;
    private final String name;
    private int enrollmentAmount = 0;
    private Set<String> enrolledStudentIds = new HashSet<>();

    public Module(String name) {
        this.id = UUID.randomUUID().toString();
        this.name = name;
    }

    public boolean enrollStudent(Student student) {
        // TODO: Implement me
    }

    public String getId() {
        return id;
    }
}

```

Exercise 1

- In `CampusSystem`, implement `findModuleById(moduleId)` returning `Optional`

```java
class CampusSystem {
    private StudentDatabase studentDatabase;
    private Set<Module> modules;

    public CampusSystem(StudentDatabase studentDatabase, Set<Module> modules) {
        this.studentDatabase = studentDatabase;
        this.modules = modules;
    }

    public Optional<Module> findModuleById(String moduleId) {
        // TODO: Implement me
    }
}

```

Question

- What happens if we swap: `Set<String> enrolledStudentIds` to `Set<Student> enrolledStudents`
- Would it still protect the uniqueness of ids?
- What to change in that case?

Why Principles?

- There are many ways to implement ideas
- Some of those are worse than others
  - e.g. Inheritance vs Composition

- In Softwaretechnik (2nd Sem): Design Patterns

OOP Principles

1. **Encapsulation**: Minimize access to classes and attributes
2. **Composition over Inheritance**
3. **Interfaces over Classes**
4. **SOLID Principles**

1. Encapsulation

- Already seen
- When possible use `private` attributes with **getter/setters**

BAD

```java
class Car {
    public String name;
}

```

GOOD

```java
class Car {
    private String name;

    public String getName() {..}

    public void setName() {..}
}

```

2. Composition

- Object contains other objects
- Models a **has-a** relation

- Inheritance: Car **is a** Vehicle
- Composition: Car **has an** Engine

Pros:

- Easier to switch objects → **Loose Coupling**
- Prevents big inheritance structures
  - Harder to switch objects!
- Extending through more composition possible

Composition > Inheritance

- Modeling a car: can play music and can move

BAD: Inheritance

```java
interface Vehicle {..}

class MusicPlayer {..}

class Car extends MusicPlayer implements Vehicle {

    void move() {..}

    void playMusic() {..}
}

```

GOOD: Composition

```java
class Engine {..}

class MusicSystem {..}

class Car {
    private MusicSystem musicSystem;
    private Engine engine;

    void move() {..}

    void playMusic() {..}
}

```

Inheritance

Problem

- Now what happens if we change the car to be electric based?

BAD: Inheritance

```java
class Car extends MusicPlayer implements Vehicle {
    void move() {
        // Have to rewrite move method
        // each time we change the engine type
    }
}

```

- Have to rewrite all implementations of Vehicle so that they all run on electric
  - → tightly coupled : hard to modify

GOOD: Composition

```java
class Engine {
    private String type;

    public void runEngine() {..}
}

class Car {
    private Engine engine;

    void move() {
        engine.runEngine();
        updateCurrentStatus();
        // ...
    }
}

// Swap engines:
Engine newEngine = new Engine();
car.setEngine(newEngine);
// Now its electric

```

3. Interfaces over Classes

- Use the least specific possible
- Already seen as:

```
// TOO SPECIFIC:
ArrayList<Pencil> pencilArrayList = new ArrayList<>();

// BETTER:
List<Pencil> pencilList = new ArrayList<>();

```

Interface

Example

```java
interface Vehicle {
    void addDriver(Person driver);

    void turnEngineOn();
}

class Car implements Vehicle {
    // Overrides..

    // Extension:
    void changeOil() {..}
}

```

BAD: too specific

```java
class Person {
    void driveCar(Car car) {
        car.addDriver(this);
    }
}

```

GOOD

```java
class Person {
    void driveVehicle(Vehicle vehicle) {
        vehicle.addDriver(this);
    }
}

```

- `Person` will only use the `addDriver()` method
- No need for any additional functionalities → only needs the `Vehicle` interface

4. SOLID

1. **S**ingle Responsibility: Each class only has 1 responsibility
2. **O**pen Closed: Components must be open to extensions but closed to changes
3. **L**iskov Substitution: Objects of superclass must be replaceable with subclasses
4. **I**nterface Segregation: Classes should not be forced to implement unnecessary methods by interfaces
5. **D**ependency Inversion: Abstracts should not depend on details

SOLID

Single Responsibility Principle

- Don't do everything in single class
- Separate into layers for modularity

- e.g. Don't have a class that parses input, executes the command and prints the result
- Instead have:
  - class for parsing
  - class for execution (business logic)
  - class for input/output

SOLID

Open Closed Principle

- Open to extensions, closed to changes
- → If need to add something, just write new code and don't change old code
- Current code should make it easier to extend and don't require changes

BAD

```java
Employee[] employees;

for (Employee employee : employees) {
    if (employee instanceof Intern) {
        employee.pay(0);
    } else if (employee instanceof Manager) {
        employee.pay(10000);
    }
}

```

GOOD

```java
Employee[] employees;

for (Employee employee : employees) {
    int salary = employee.getSalary();
    employee.pay(salary);
}

```

- This way if you add new roles → don't have to add new else branch

SOLID

Liskov Substitution Principle

- Superclasses should be replaceable with subclasses

```
void doSomething(Number number) {
    // ...
}

Number number = 10;
doSomething(number);

// Should not cause any trouble:
Integer integer = 10;
doSomething(integer);

```

SOLID

Liskov Substitution Principle

- Problem starts when the relation is wrong/broken
- LSP is violated when a subclass cannot honor the behavioral contract of its superclass — even if it “is-a” conceptually

```java
class Rectangle {
    protected int width;
    protected int height;

    public void setWidth(..) {..}

    public void setHeight(..) {..}

    public int area() { return width * height; }
}

void resize(Rectangle r) {
    r.setWidth(5);
    r.setHeight(10);
    System.out.println(r.area());
}
```

```java
class Square extends Rectangle {
    @Override
    public void setWidth(int width) {
        this.width = width;
        this.height = width
    }

    @Override
    public void setHeight(int height) {
        this.height = height;
        this.width = height;
    }
}

resize(new Rectangle());  // correct: 50
resize(new Square());  // false: 100
```

SOLID

Interface Segregation Principle

- If we have interface Bird that can **fly**
- Problem: Penguin implements Bird but shouldn't implement fly

```java
interface Bird {
    void fly();
}

class Penguin implements Bird {
    @Override
    public void fly() {
        // Do nothing
        return;
    }
}

```

- Instead the interfaces should be separated

SOLID

Dependency Inversion Principle

- We have a `UserService` that depends on some `UserRepository`
- Without dependency inversion it looks like this:

```
class UserService {
    private ElasticDB database = new ElasticDB();
    private UserRepository userRepo = new UserRepo(database);

    void doSomething() {
        userRepository.add(..);
    }
}

```

- Now what happens if we want to change the repository to use MongoDB instead?
- Have to replace the database in service class as well

SOLID

Dependency Inversion Principle

- With dependency inversion it would look like this:

```
class UserService {
    private Repository repo;

    public UserService(Repository repository) {
        this.repository = repository;
    }

    // ...
}

```

- Now we can instead just pass in another repository and don't use the class itself
- We are using the interfaces and can swap the pieces to other classes
- Don't have to modify the service class while doing so

Conclusion

- Java API
- OOP Principles

Exercise 1

```java
class StudentDatabase {
    private final Map<String, Student> students = new HashMap<>();

    public Optional<Student> getStudent(String id) {
        Student student = students.get(id);
        if (student == null) {
            return Optional.empty();
        }
        return Optional.of(student);
    }

    public boolean addStudent(Student student) {
        String id = student.getId();

        if (students.containsKey(id)) {
            return false;
        }
        students.put(id, student);
        return true;
    }
}

```

```java
class Module {
    // ...

    public boolean enrollStudent(Student student) {
        if (enrolledStudentIds.contains(student.getId())) {
            return false;
        }

        boolean result = enrolledStudentIds.add(student.getId());
        if (result) {
            enrollmentAmount++;
        }
        return result;
    }

    // ...
}

```

Exercise 1

```java
class CampusSystem {
    private StudentDatabase studentDatabase;
    private Set<Module> modules;

    public CampusSystem(StudentDatabase studentDatabase, Set<Module> modules) {
        this.studentDatabase = studentDatabase;
        this.modules = modules;
    }

    public Optional<Module> findModuleById(String moduleId) {
        for (Module module : modules) {
            if (module.getId().equals(moduleId)) {
                return Optional.of(module);
            }
        }
        return Optional.empty();
    }
}

```